

Webprogrammierung: JavaScript Grundlagen und Datenstrukturen

Fachlehrer Stefan Sauer
THWS Geovisualisierung

Javascript Grundlagen

**Wenn html das Grundgerüst und css
das Design ist, dann ist Javascript
das was Leben einhaucht!**

Einführung & Moderne Sprachgrundlagen

Übersicht:

- Woher kommt JavaScript
- Was javascript (nicht) kann
- Sicherheit
- ECMAScript
- Entwicklertools

Woher kommt JavaScript?

- Erfunden 1995 von "Netscape"
- später später mit ECMA
(European Computer Manufacturers Association) weiter entwickelt
- Javascript ist nicht gleich Java!

Was Javascript kann

- kann komplette Anwendungen entwickeln
- kann grafische Objekte manipulieren und animieren
- kann Elemente dynamisieren
- kann Daten zwischen Browser und Webserver austauschen, ohne dass Seite neu geladen werden muss
- kann Daten auf dem Client speichern
- kann lokalisierte Informationen anzeigen
- kann Daten des Clientrechners auswerten und Seiten und Inhalte an die Gegebenheiten des Besuchers anpassen

Was Javascript nicht kann

- keine Datenbank abfragen
- keine Dateien auf dem Server speichern oder organisieren
- kann keine Programme auf dem Server ausführen
- kann keine Informationen aus anderen Webseiten abrufen, da diese mit CORS-Mechanismen geschützt sein können.
- kann Standardmäßig nicht direkt auf das Dateisystem des Computers zugreifen (aus Sicherheitsgründen)
- kann nicht direkt auf die Hardware des Computers zugreifen (aus Sicherheitsgründen)

Javascript und Sicherheit

Früher war Javascript ein Sicherheitsrisiko.

Heute ist es das nicht mehr. Die Programmiersprache wird von den Herstellern der Browser stark reglementiert.

**Javascript läuft also nur im
"Sandkasten" (Sandbox) des
Browsers!**

ECMAScript vs. JavaScript-Engines

- **ECMAScript (ES):** Der offizielle **Bauplan** (Standard & Spezifikation)
 - Legt Regeln, Syntax und neue Features fest (z. B. ES6, `const` / `let` , Arrow Functions).
 - Sagt, *wie* JavaScript geschrieben werden muss.
- **JS-Engine:** Der **Motor** (die Software im Browser)
 - Liest den JavaScript-Code und übersetzt ihn in Maschinencode (Ausführung).
 - Setzt den ECMAScript-Bauplan in der Praxis um.
- **Die bekanntesten Engines:**
 - **V8:** Google Chrome, Node.js, Microsoft Edge, Brave
 - **JavaScriptCore (Nitro):** Apple Safari (iOS / macOS)
 - **SpiderMonkey:** Mozilla Firefox

ECMAScript ist das **Rezept**,
die JS-Engine ist der **Koch**,
der das **Gericht** zubereitet!

**Javascript wird zur Laufzeit von der
JS-Engine des Browsers übersetzt,
interpretiert und ausgeführt.**

Entwicklertools & Debugging

Entwicklertools der Browser (Console, Sources, Debugging)

- **Überblick:** Alle modernen Browser (Chrome, Firefox, Edge, Safari) bieten integrierte Entwicklerwerkzeuge (DevTools).
- **Zugriff:** Taste `F12` oder `Rechtsklick -> Element untersuchen`.
- **Die wichtigsten Tabs:**
 - **Console:** Interaktives Terminal für JS-Befehle, Fehlermeldungen & Ausgaben (`console.log()` , `console.error()`).
 - **Sources:** Einsicht in geladene `.js` -Dateien, Quellcode-Inspektion & Breakpoints.
 - **Network:** Überwachung aller Netzwerkanfragen (z. B. API-Aufrufe/Fetch).

**Die DevTools sind das Röntgengerät
des Webentwicklers – sie machen
die unsichtbare Laufzeit im Browser
sichtbar!**

Breakpoint-Debugging: Code anhalten & untersuchen

- **Warum Breakpoints statt nur `console.log()` ?**
 - `console.log()` vermüllt den Code und zeigt nur statische Momentaufnahmen.
 - Breakpoints halten das Programm zur Laufzeit in der JS-Engine an!
- **Workflow im Sources-Tab:**
 1. Klick auf die Zeilennummer im Code setzt einen **Breakpoint**.
 2. Die Ausführung stoppt sofort, wenn die Zeile erreicht wird.
 3. **Scope & Inspector:** Live-Einsicht in alle aktuellen Variablenwerte und den Aufrufstapel (Call Stack).
- **Steuerung im Debugger:**
 - **Step Over (*F10*)**: Zur nächsten Zeile springen.
 - **Step Into (*F11*)**: In die Funktionsausführung hineinspringen.
 - **Resume (*F8*)**: Normale Programmausführung fortsetzen.

Das `debugger;`-Statement & Konsole-Fehler verstehen

- **Code-Stopp per Befehl:**
 - Das Statement `debugger;` im JS-Code wirkt wie ein fest eingebauter Breakpoint (sofern die DevTools geöffnet sind).
- **Die 3 häufigsten JS-Fehler in der Console lesen:**
 - **ReferenceError:** Variable ist nicht deklariert oder falsch geschrieben (`x is not defined`).

Praxis-Tipp: Ein Klick auf die Zeilennummer rechts neben der Fehlermeldung in der Konsole springt direkt zur Fehlerquelle im Sources-Tab!

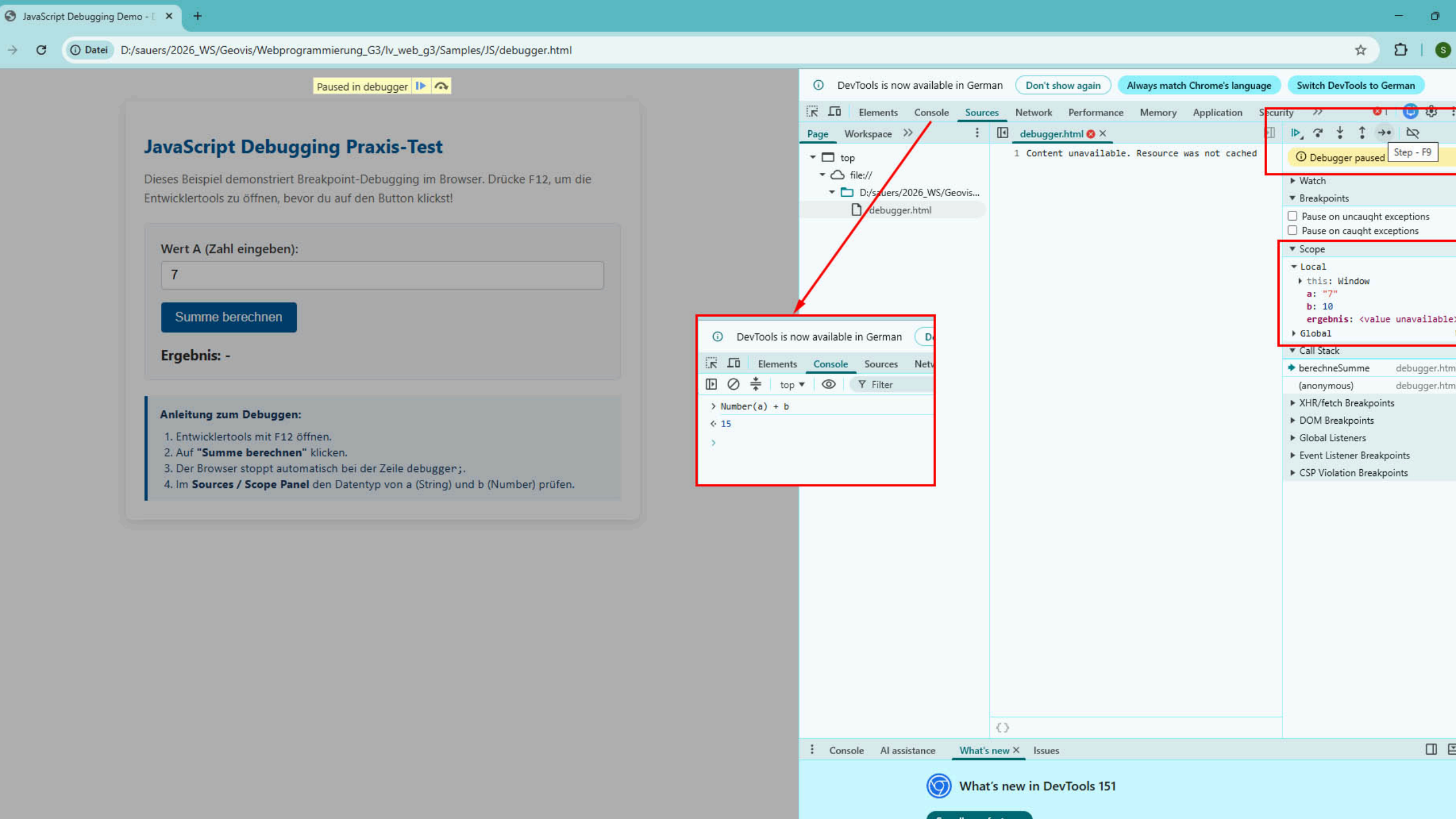
Praxis-Beispiel zum Nachbasteln: Bug-Hunting

```
<button id="calcBtn">Summe berechnen</button>
<p id="result"></p>

<script>
  function berechneSumme(a, b) {
    debugger; // Stoppt automatisch bei geöffneten DevTools!
    return a + b;
  }
  document.getElementById("calcBtn").addEventListener("click", () => {
    let eingabeA = "5"; // Achtung: Wert aus Input-Feld ist ein String!
    let eingabeB = 10;
    let summe = berechneSumme(eingabeA, eingabeB);
    document.getElementById("result").innerText = "Ergebnis: " + summe;
  });
</script>
```

Schritt-für-Schritt: Vorgehensweise beim Debuggen

- **1. DevTools öffnen:** Seite laden und `F12` (oder `Rechtsklick -> Untersuchen`) drücken.
- **2. Aktion ausführen:** Klick auf den Button "Summe berechnen".
 -  Browser stoppt automatisch bei `debugger;` in der Funktion `berechneSumme()`.
- **3. Variablen inspizieren (Sources-Tab → Scope):**
 - Fahre mit der Maus über `a` und `b` oder prüfe das **Scope-Panel**.
 - Erkenne den Typ: `a = "5"` (String) vs. `b = 10` (Number).
- **4. Fehlerursache:** `a + b` führt wegen String-Konkatenation zu `"510"` statt `15`.
- **5. Live-Fix in der Console testen:** In der Konsole `Number(a) + b` ausführen → `15`.



Der didaktische Debugging-Workflow

- **1. Konsole prüfen:** Gibt es rote Fehlermeldungen (z. B. `TypeError`)?
- **2. Breakpoint setzen:** Zeilennummer im *Sources-Tab* anklicken oder `debugger;` einbauen.
- **3. Variablenzustand durchleuchten:** Werte im *Scope-Fenster* zur Laufzeit analysieren.
- **4. Code Schritt für Schritt ausführen:** Mit *F10* (Step Over) den Ablauf kontrollieren.
- **5. Fehler beheben:** Typumwandlung ergänzen (`Number(...)`) & Code korrigieren.

**Debuggen ist systematische
Hypothesenprüfung – nicht Raten!**

Einbindung von Javascript

Saubere Einbindung im HTML: `<script defer>` und ES-Module (`type="module"`)

- **Drei Wege zur Einbindung:** Inline (veraltet), Intern (`<script>`), Extern (`<script src="...">`).

Die 3 Arten der JavaScript-Einbindung

1. Inline-Event-Handler (Veraltet / No-Go):

- `<button onclick="alert('Hallo!')">Klick</button>`
-  Vermischt HTML & Logik, schwer wartbar, Sicherheitsrisiken.

2. Internes Skript (`<script>` -Tag im HTML):

- `<script> console.log("Hallo Welt"); </script>`
-  Gut für kleine Test-Skripte & Demos.  Nicht wiederverwendbar.

3. Externes Skript (Standard & Best Practice):

- `<script src="app.js"></script>`
-  **Saubere Trennung**, Browser-Caching, wiederverwendbar.

Vergleich: Vor- und Nachteile der Einbindung

Methode	Beispiel	Vorteile	Nachteile
Inline (<i>veraltet</i>)	<code>onclick="..."</code>	Schnell für 1 - Zeiler	Vermischt HTML & JS, kein Caching, Sicherheitsrisiko
Intern	<code><script>... </script></code>	Kein Extra - HTTP - Request	Kein Caching für JS, macht HTML unübersichtlich
Extern (<i>Standard</i>)	<code><script src="..."> </script></code>	Wiederverwendbar, Browser - Caching , saubere Struktur	Benötigt Netzwerk - Request (gecached)

Grundregel:

Trenne immer:

- Inhalt (HTML)
- Layout (CSS)
- Verhalten (JS)

Hinweis:

Veraltete Attribute wie **language="javascript"** werden nicht mehr genutzt.

Ladeverhalten optimieren: Das `defer`-Attribut

- **Klassisches Verhalten (ohne Attribut):**
 - Skript lädt & führt sofort aus → **blockiert das Parsen des HTML-DOMs.**
- **Die moderne Lösung: `<script defer src="app.js">`**
 - `defer` (**Aufschieben**): Lädt das Skript im Hintergrund herunter.
 - Ausführung erfolgt **erst, nachdem das HTML vollständig aufgebaut (geparst) ist.**
 - Garantiert die korrekte Ausführungsreihenfolge mehrerer Skripte.
- **Unterschied zu `async` :**
 - `async` führt das Skript sofort nach dem Download aus (gut für unabhängige Analyse-Tools).

Best Practice:
Platziere externe Skripte im <head>
und nutze immer defer!

Modern & Modular: ES-Module (`type="module"`)

- **Einbindung:**

- `<script type="module" src="app.js"></script>`

- **Die wichtigsten Eigenschaften von ES-Modulen:**

- `import` & `export` : Code lässt sich in wiederverwendbare Dateien aufteilen.

- **Automatisches** `defer` : Modul-Skripte werden standardmäßig aufschiebend geladen.

- **Eigener Scope:** Variablen verschmutzen nicht den globalen `window` -Namensraum.

- **Strict Mode:** Läuft automatisch im modernen `"use strict"` -Modus.

```
// math.js (Funktion exportieren)
export function summe(a, b) { return a + b; }
```

```
// app.js (Funktion importieren & nutzen)
import { summe } from './math.js';
```

Praxis-Beispiel: Modularer Code (calculator.js)

```
// calculator.js (Hilfsmodul)

// Privat: Nicht exportiert -> Von außen unsichtbar!
function formatierErgebnis(wert) {
    return `[Ergebnis: ${wert}]`;
}

// Öffentlich: Durch 'export' für andere Dateien verfügbar
export function addieren(a, b) {
    return formatierErgebnis(Number(a) + Number(b));
}

export function subtrahieren(a, b) {
    return formatierErgebnis(Number(a) - Number(b));
}
```

Hauptsript & HTML-Einbindung (main.js & HTML)

```
// main.js (Hauptprogramm)
import { addieren, subtrahieren } from './calculator.js';

document.getElementById('addBtn').addEventListener('click', () => {
  const valA = document.getElementById('numA').value;
  const valB = document.getElementById('numB').value;
  document.getElementById('output').innerText = addieren(valA, valB);
});
```

```
<!-- HTML-Einbindung: type="module" aktiviert import/export -->
<script type="module" src="main.js"></script>
```

Wichtige Stolperfallen bei ES-Modulen

- **1. Pfadangabe beim Import:**

- ❌ `import { addieren } from 'calculator.js';` (Fehler im Browser!)
- ✅ `import { addieren } from './calculator.js';` (Relativer Pfad mit `./` Pflicht!)

- **2. Browser-Sicherheitsregel (CORS / `file://`):**

- Öffnen per Doppelklick (`file:///...`) wird aus Sicherheitsgründen blockiert.
- **Lösung:** Ausführung über lokalen Server (z. B. VS Code Extension *Live Server*).

**Beispiel zum Ausprobieren: Liegt
spielfertig unter `Samples/JS/modules-`
`demo.html` .**

Variablen und Datentypen

Übersicht:

- Variablendeklaration (`const` , `let`)
- 7 primitive Typen
- strikte Vergleiche (`===`).

Variablen deklarieren: `const` vs. `let` (vs. `var`)

- **`const` (Standard):**
 - Für Variablen, deren Referenz **nicht verändert** wird.
 - Verhindert unbeabsichtigtes Überschreiben (`const pi = 3.14159;`).
- **`let` (Veränderlich):**
 - Für Variablen, deren Wert sich **später ändert** (z. B. Zähler in Schleifen).
 - Gültig nur im jeweiligen Code-Block (*Block Scope* `{ ... }`).
- **✗ Entfall: Kein `var` mehr nutzen!**
 - `var` hat keinen Block-Scope und leidet unter *Hoisting* (Hochziehen von Variablen). Führt in der Praxis zu schwer auffindbaren Bugs.

Was ist Hoisting ("Hochziehen")?

Die JS-Engine zieht Deklarationen vor der Ausführung gedanklich an den Anfang des Scopes:

- **var (Gefährlich):**
 - Deklaration wird hochgezogen, Zuweisung bleibt stehen
 - Zugriff vor Zeile ergibt `undefined` (kein Fehler!).
- **let & const (Sicher):**
 - Liegen vor der Zuweisungszeile in der **Temporal Dead Zone (TDZ)**
 - Zugriff erzeugt einen sauberen `ReferenceError` .

```
console.log(a); // undefined (mit var)
var a = 5;

console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 10;
```

**Regel: Nutze immer *const*. Erst wenn
der Wert neu zugewiesen werden
muss, ändere es zu *let*!**

Weitere Informationen unter:

<https://www.cancode.de/blog-article/die-eigenheiten-von-var-let-und-const-in-javascript>

Die 7 primitiven Datentypen in JavaScript

Primitive Daten sind **unveränderlich** (*immutable*) und werden direkt als Wert gespeichert:

1. **string** : Textdaten (`"Hallo"` , `'Welt'`)
2. **number** : Zahlen & Kommazahlen (`42` , `3.14` , `NaN`)
3. **boolean** : Wahrheitswerte (`true` / `false`)
4. **undefined** : Variable deklariert, aber noch kein Wert zugewiesen (`let x;`)
5. **null** : Bewusste Abwesenheit eines Werts (`const daten = null;`)
6. **bigint** : Beliebige große Ganzzahlen (`9007199254740991n`)
7. **symbol** : Eindeutiger Identifikator (`Symbol("id")`)

Typprüfung im Code mit dem Operator `typeof` (z. B. `typeof "Hallo"` → `"string"`).

Vergleiche: Strikte (===) vs. Lose (==) Gleichheit

- **Lose Gleichheit (==): Gefährlich!**
 - Führt eine **automatische Typkonvertierung** (*Type Coercion*) durch.
 - "5" == 5 → true | 0 == false → true | "" == 0 → true
- **Strikte Gleichheit (===): Best Practice!**
 - Vergleicht **Wert UND Datentyp** (ohne implizite Typumwandlung).
 - "5" === 5 → false (*String vs. Number*)
 - 5 === 5 → true

Merksatz: Verwende in JavaScript immer === und !==, um unerwartete Typ-Bugs zu vermeiden!

Praxis-Check: Datentypen & Vergleiche in der Konsole

```
const name = "Anna";    // String
let alter = 22;          // Number
let istStudent = true;  // Boolean

console.log(typeof name); // "string"
console.log(typeof alter); // "number"

// Vergleichstest:
console.log("22" == alter); // true  (Lose Gleichheit -> Typumwandlung!)
console.log("22" === alter); // false (Strikte Gleichheit -> Sicherer Standard!)
```

Sonderfall / Kuriosität: `typeof null` **gibt**
historisch bedingt `"object"` **zurück**
(bekannter JS-Bug seit 1995).

String-Verarbeitung & Template Literals

Strings erstellen und verarbeiten (Eigenschaften & Methoden)

Übersicht:

-Nützliche String-Methoden

- Template Literals (Backticks)
- mehrzeilige Strings.

Strings verarbeiten: Wichtige Eigenschaften & Methoden

Strings sind Zeichenketten mit vielen eingebauten Hilfsmitteln:

- `text.length` : Anzahl der Zeichen im String (`"Hallo".length` → `5`).
- `text.toUpperCase()` / `.toLowerCase()` : Wandelt den Text um (`"hi".toUpperCase()` → `"HI"`).
- `text.includes("web")` : Prüft, ob ein Teiltext enthalten ist (liefert `true` / `false`).
- `text.trim()` : Entfernt Leerzeichen am Anfang und Ende.
- `text.slice(0, 4)` : Schneidet einen Teilstring heraus (`"JavaScript".slice(0, 4)` → `"Java"`).

Template Literals: Die Macht der Backticks (`)

Mit Backticks (``...``) werden Strings dynamisch und lesbar:

- **String-Interpolation (`${...}`):**
 - Bette Variablen & Ausdrücke direkt im Text ein: ``Preis: ${preis * 1.19} €``.
- **Mehrzeilige Strings (Multiline):**
 - Zeilenumbrüche können direkt im Code getippt werden (kein `\n` erforderlich!).

```
const name = "Alex";
const punkte = 42;

// Dynamisch & Mehrzeilig:
const nachricht = `Hallo ${name},
willkommen zurück!
Deine Punkte: ${punkte}`;
```

Vorher vs. Nachher: `+`-Konkatenation vs. Backticks

- ✗ **Veraltet (unübersichtliche `+`-Verknüpfung):**

```
st info = "User: " + vorname + " " + namenname + " (" + alter + " Jahre)";
```

- ✓ **Modern & Sauber (Template Literals):**

```
st info = `User: ${vorname} ${namenname} (${alter} Jahre)`;
```

Regel: Nutze für jeden zusammengesetzten oder mehrzeiligen Text immer Template Literals (Backticks)!

Kontrollstrukturen, Funktionen & Scope

Kontrollstrukturen & Entscheidungen

Übersicht:

- Verzweigungen (`if/else`)
- Mehrfachauswahl (`switch/case`)
- ternärer Operator (`? :`).

Verzweigungen: `if`, `else if` und `else`

Steuere den Programmfluss basierend auf Wahrheitswerten (`true` / `false`):

```
const alter = 18;

if (alter >= 18) {
  console.log("Volljährig: Zutritt gestattet.");
} else if (alter >= 16) {
  console.log("Zutritt in Begleitung gestattet.");
} else {
  console.log("Kein Zutritt.");
}
```

- **Ablauf:** Bedingungen werden sequenziell von oben nach unten geprüft.
- **Logik:** Sobald eine Bedingung `true` ergibt, wird dieser Block ausgeführt und der Rest übersprungen.

Mehrfachauswahl: `switch` / `case`

```
const rolle = "admin";

switch (rolle) {
  case "admin":
    console.log("Vollzugriff gewährt.");
    break; // Verhindert das Durchfallen in den nächsten Case!
  case "editor":
    console.log("Inhalte bearbeiten erlaubt.");
    break;
  default:
    console.log("Standard: Nur Lesezugriff.");
}
```

Wichtig: Ohne *break*; führt JS automatisch die darunterliegenden *case-Blöcke* aus (*Fallthrough*).

Der Ternäre Operator (*Bedingung ? Wert1 : Wert2*)

Kompakte Kurzschreibweise für einfache `if / else`-Entscheidungen, die einen Wert zurückliefern:

```
const alter = 20;

// Klassisch (if / else):
let statusText;
if (alter >= 18) { statusText = "Erwachsen"; } else { statusText = "Minderjährig"; }

// Elegant mit Ternärem Operator:
const statusText = alter >= 18 ? "Erwachsen" : "Minderjährig";
```

**Best Practice: Nutze den ternären
Operator für prägnante Einzeiler.
Vermeide Schachtelungen!**

Praxis-Beispiel: Kombination aller 3 Entscheidungen

```
// 1. IF / ELSE IF / ELSE: Bereichsprüfung (Alter)
if (alter < 6) { basispreis = 0; }
else if (alter < 18) { basispreis = 8.00; } // Ermäßigt
else { basispreis = 12.00; }              // Standard

// 2. SWITCH / CASE: Exakter Abgleich (Wochentag)
switch (tag) {
    case "Dienstag": endpreis = basispreis * 0.8; break; // Kinotag
    case "Mittwoch": endpreis = basispreis - 2.0; break; // Studententag
    default: endpreis = basispreis; break;
}

// 3. TERNÄRER OPERATOR: Einzeiler für Statusmeldung
const seatText = isVip ? "VIP-Lounge (+ 5 €)" : "Standard-Platz";
```

Entscheidungs-Matrix: Wann nutzt man was?

Kontrollstruktur	Ideal für...	Beispiel - Usecase
<code>if / else</code>	Bereichstests (<code><</code> , <code>></code> , <code>>=</code>) & komplexe Bedingungen (<code>&&</code> , <code> </code>)	Altersprüfungen, Notenberechnung
<code>switch / case</code>	Exakte Wertabgleiche einer Variable bei vielen Alternativen	Wochentage, Benutzerrollen, Menüauswahl
Ternär (<code>? :</code>)	Kompakte Zuweisungen auf Basis einer Ja/Nein - Frage	Status - Texte, Standardwerte setzen

Beispiel zum Ausprobieren:

Liegt spielfertig unter

`Samples/JS/decisions-demo.html` .

Schleifen & moderne Iteration

Klassische Schleifen:

- for
- while
- do-while

Moderne Iteration:

- for...of für Werte
- for...in für Objekteigenschaften

Klassische Schleifen: `for`, `while` & `do-while`

- **Zählschleife (`for`)**: Bekannte Anzahl an Durchläufen.

```
(let i = 0; i < 3; i++) {  
  console.log(`Durchlauf: ${i}`); // 0, 1, 2  
}
```

- **Kopfgesteuerte Schleife (`while`)**: Prüft Bedingung **vor** dem Durchlauf.

```
countdown = 3;  
while (countdown > 0) { console.log(countdown--); }
```

- **Fußgesteuerte Schleife (`do...while`)**: Wird **mindestens 1-mal** ausgeführt (Prüfung danach).

```
eingabe;  
do { eingabe = "ja"; } while (eingabe !== "ja");
```

Moderne Iteration: `for...of` vs. `for...in`

- **`for...of` (Für Werte in Arrays / Iterables):**

→ Iteriert direkt über die **Elemente** (sauber & ohne Index-Stress).

```
st staedte = ["Würzburg", "Nürnberg", "München"];
for (const stadt of staedte) {
  console.log(stadt); // "Würzburg", "Nürnberg", ...
}
```

- **`for...in` (Für Schlüssel / Objekteigenschaften):**

→ Iteriert über die **Keys** eines Objekts.

```
st user = { name: "Anna", rolle: "Admin" };
for (const key in user) {
  console.log(`${key}: ${user[key]}`); // "name: Anna", ...
}
```


Faustregel: *for...of* für Arrays (Werte),
for...in für Objekte (Keys)!

Schleifensteuerung: `break` & `continue`

Steuere den Ablauf innerhalb einer laufenden Schleife gezielt:

- **`break` (Sofortiger Abbruch):**

→ Beendet die Schleife komplett und springt dahinter.

```
(let i = 1; i <= 10; i++) {  
  if (i === 4) break; // Stoppt die Schleife bei 4!  
  console.log(i); // Gibt 1, 2, 3 aus  
}
```

- **`continue` (Überspringen des aktuellen Durchlaufs):**

→ Bricht nur den aktuellen Durchgang ab und springt zum nächsten.

```
(let i = 1; i <= 4; i++) {  
  if (i === 3) continue; // Überspringt nur die 3!  
  console.log(i); // Gibt 1, 2, 4 aus  
}
```

Übersicht & Best Practices der Iteration

Schleife	Liest...	Haupt - Anwendungsfall
<code>for</code>	Index (<code>i++</code>)	Feste Anzahl / komplexe Zählschritte
<code>while</code>	Bedingung (<code>true</code>)	Warten auf Ereignis / dynamischer Zustand
<code>for...of</code> (Standard)	Werte	Arrays & Kollektionen durchlaufen
<code>for...in</code>	Keys (Schlüssel)	Objekt - Eigenschaften inspizieren

Best Practice: Nutze in modernem JavaScript für Arrays fast immer *for...of* – das vermeidet Typfehler und ist am lesbarsten.

Praxis-Check: Schleifen & Iteration im Einsatz

- **Ausführbare Demodatei:** `Samples/JS/loops-demo.html`
- **Interaktiver Test im Browser:**
 1. `for...of` : Liest Werte eines Arrays direkt aus (`const frucht of fruechte`).
 2. `for...in` : Inspiziert Schlüssel-Wert-Paare eines Objekts (`const key in produkt`).
 3. `while` : Zählt einen Lagerbestand herunter, solange er größer als `0` ist.
 4. `break` & `continue` : Demonstriert das Überspringen (`continue`) und den Abbruch (`break`).

Ausprobieren:

Öffne *Samples/JS/loops-demo.html* im Browser und teste die Buttons mit Live-Konsolenausgabe! Code und dessen Funktion lesen und verstehen.

Funktionen & Arrow Functions

Übersicht:

- Deklarationen vs. Arrow Functions
- Default- & Rest-Parameter
- Scope & Closures.

Function Declaration vs. Function Expression

- **Function Declaration (Klassische Funktion):**

→ Wird per *Hoisting* an den Anfang des Scopes gezogen → vor ihrer Deklaration aufrufbar.

```
function gruss(name) {  
    return `Hallo ${name}!`;  
}
```

- **Function Expression (Funktionsausdruck):**

→ Funktion wird in einer Variable gespeichert → erst **ab der Zuweisung** aufrufbar.

```
const gruss = function(name) {  
    return `Hallo ${name}!`;  
};
```

Unterschied:

**Declarations sind global/funktional
verfügbar**

**Expressions folgen den normalen
Variablen-Regeln (*const*).**

Modern & Kompakt: Arrow Functions (=>)

Die prägnanteste Schreibweise für Funktionen in modernem JavaScript:

```
// Standard-Schreibweise:  
const quadrat = (x) => { return x * x; };  
  
// Kurzschreibweise (Impliziter Return bei Einzeilern):  
const quadrat = x => x * x;
```

- **Die 3 Kern-Merkmale von Arrow Functions:**
 1. **Kompakte Syntax:** Kein `function`-Keyword erforderlich.
 2. **Implizites `return`:** Bei einzeiligen Ausdrücken ohne geschweifte Klammern `{}`.
 3. **Lexikalisches `this`:** Erbt das `this` aus dem umgebenden Kontext (ideal für Event-Handler).

Standard-Parameter & Rest-Parameter (...args)

- **Standard-Parameter (Default Parameters):**

→ Beugt `undefined`-Fehlern vor, indem Standardwerte hinterlegt werden.

```
function erstelleUser(name = "Anonym", rolle = "Gast") {  
    return `${name} (${rolle})`;  
}
```

- **Rest-Parameter (...args statt veraltetem arguments):**

→ Bündelt beliebig viele Argumente in ein **echtes Array**.

```
function summiere(...zahlen) {  
    return zahlen.reduce((summe, n) => summe + n, 0);  
}  
summiere(5, 10, 15); // 30
```

Scope, Lexical Environment & Closures

- **Global Scope vs. Block Scope:**

→ `const` / `let` sind streng im umschließenden Block `{ ... }` geschützt.

- **Was ist eine Closure (Funktionsabschluss)?**

→ Eine innere Funktion hat Zugriff auf Variablen ihres **äußeren Entstehungs-Scopes** – selbst nachdem die äußere Funktion beendet ist!

```
function erstelleZaehler() {  
    let count = 0; // Kapselung im Scope der Closure  
    return () => ++count;  
}  
  
const zaehler = erstelleZaehler();  
console.log(zaehler()); // 1  
console.log(zaehler()); // 2
```

Vergleich & Best Practices für Funktionen

Merkmal	Function Declaration	Arrow Function (=>)
Syntax	<code>function name() {}</code>	<code>const name = () => {}</code>
Hoisting	✅ Ja (überall aufrufbar)	❌ Nein (erst nach Zeile verfügbar)
<code>this</code> -Bindung	Dynamisch (hängt vom Aufrufer ab)	Lexikalisch (erbt vom Umfeld)
Impliziter Return	❌ Nein (benötigt <code>return</code>)	✅ Ja (bei Einzeiler ohne <code>{}</code>)

Best Practice: Nutze Arrow Functions für Helper, Array-Methoden & Event-Handler. Nutze Function Declarations für zentrale Skript-Funktionen.

Praxis-Check: Funktionen & Arrow Functions im Einsatz

- **Ausführbare Demodatei:** `Samples/JS/functions-demo.html`
- **Gezeigte Funktions-Konzepte:**
 1. **Default-Parameter:** `formatierPreis(betrag, waehrung = "€")` schützt vor `undefined`.
 2. **Rest-Parameter:** `berechneSumme(...preise)` fasst beliebige Werte in ein Array zusammen.
 3. **Arrow Functions (`=>`):** Implizite Rückgabe bei `.map(brutto => (brutto / 1.19).toFixed(2))`.
 4. **Closures:** `erstelleWarenkorbTracker()` kapselt den Zählerstand privat im Scope.

Ausprobieren: Öffne `Samples/JS/functions-`
`demo.html` **im Browser und teste die**
Interaktionen!

Komplexe Datenstrukturen (Objekte & Arrays)

Übersicht:

- Moderne Objektorientierung mit `class`
- Schlüssel-Wert-Speicher: Objekte & JSON-Notation
- Moderne Array-Methoden
- Destructuring & Spread-Operator

Objekte & JSON-Notation (Schlüssel-Wert-Speicher)

- **Objekte in JavaScript:**

→ Sammlungen von Eigenschaften (*Key-Value Pairs*).

```
const user = { name: "Anna", alter: 22, geovisStudent: true };  
console.log(user.name);      // Punkt-Notation: "Anna"  
console.log(user["alter"]);  // Klammer-Notation: 22
```

- **JSON (JavaScript Object Notation):**

→ Standardformat für den Datenaustausch im Web (Web-APIs & Server).

→ **JSON.stringify(obj)** : Wandelt ein JS-Objekt in einen JSON-String um.

→ **JSON.parse(jsonString)** : Wandelt einen JSON-String zurück in ein JS-Objekt.

Moderne Objektorientierung mit `class`

JS-Version ES6 führte die saubere `class` -Syntax ein (ersetzt alte Prototypen-Konstrukturen):

```
class Person {  
  #geheimnis = 42; // Privates Feld (#) -> von außen nicht direkt lesbar!  
  
  constructor(name, rolle) {  
    this.name = name; // Instanz-Variable  
    this.rolle = rolle;  
  }  
  
  vorstellen() { // Methode  
    return `Ich bin ${this.name} (${this.rolle}).`;   
  }  
}  
  
const p = new Person("Max", "Dozent");  
console.log(p.vorstellen()); // "Ich bin Max (Dozent)."
```

Vererbung mit `extends` und `super`

Klassen können Eigenschaften und Methoden von Elternklassen erben:

```
class Student extends Person {
  constructor(name, matrikelnummer) {
    // super() ruft den Konstruktor der Elternklasse (Person) auf:
    super(name, "Student");
    this.matrikelnummer = matrikelnummer;
  }

  lernen() {
    return `${this.name} (Matrikel: ${this.matrikelnummer}) lernt JS.`;
  }
}

const s = new Student("Lisa", 632217);
console.log(s.vorstellen()); // Von Person geerbt!
console.log(s.lernen());    // Eigene Methode
```


Praxis-Check: Objekte & JSON im Einsatz

- **Ausführbare Demodatei:** `Samples/JS/objects-json-demo.html`
- **Gezeigte Kernkonzepte im Code:**
 1. **JS-Objekt & Zugriffe:** Punkt-Notation (`student.name`) vs. Klammer-Notation (`student['id']`) & verschachtelte Objekte.
 2. `JSON.stringify(obj, null, 2)` : Wandelt ein JS-Objekt in ein lesbares JSON-String-Format um.
 3. `JSON.parse(jsonText)` : Parst empfangenen JSON-Text vom Server zurück in ein echtes JS-Objekt für Berechnungen.
 4. **Objekt-Methoden & `this`** : Funktions-Methoden innerhalb von Objekten mit Zugriff auf eigene Eigenschaften via `this` .

Ausprobieren:

Öffne `Samples/JS/objects-json-demo.html` im Browser und teste die Live-Konsole!

Arrays erstellen & Grundlagen (Array-Literale `[]`)

Arrays sind **geordnete Listen** von Elementen mit 0-basiertem Index:

```
// 1. Erstellung per Array-Literal [] (Best Practice!):  
const staedte = ["Würzburg", "Nürnberg", "München"];  
  
// 2. Zugriff über den 0-basierten Index & Länge prüfen:  
console.log(staedte[0]);    // "Würzburg" (Erstes Element)  
console.log(staedte.length); // 3 (Gesamtzahl der Elemente)  
  
// 3. Element am Index verändern:  
staedte[1] = "Bamberg";    // Ersetzt "Nürnberg"
```

- **Eigenschaft:** Arrays können beliebige Datentypen mischen (Strings, Zahlen, Objekte).
- **Merksatz:** Nutze immer `[]` statt `new Array()`.

Moderne Array-Methoden (Funktionaler Stil)

Methoden zur sauberen Datenverarbeitung ohne manuelle `for`-Schleifen:

- **`map()` (Transformieren):** Wandelt jedes Element um.
→ `[1, 2, 3].map(x => x * 2)` → `[2, 4, 6]`
- **`filter()` (Filtern):** Behält nur Elemente, die die Bedingung erfüllen.
→ `[10, 5, 20].filter(x => x >= 10)` → `[10, 20]`
- **`reduce()` (Aggregieren):** Berechnet einen einzelnen Gesamtwert.
→ `[10, 20, 30].reduce((sum, n) => sum + n, 0)` → `60`
- **`find()` & `includes()`:** Sucht Ersttreffer (`find`) oder prüft Existenz (`includes`).

Destructuring Assignment (Werte entpacken)

Kompaktes Extrahieren von Werten aus Objekten und Arrays in eigene Variablen:

```
// 1. Objekt-Destructuring:  
const user = { name: "Sarah", stadt: "Würzburg", alter: 24 };  
const { name, stadt } = user; // Erzeugt Variablen 'name' und 'stadt'  
console.log(`${name} aus ${stadt}`); // "Sarah aus Würzburg"  
  
// 2. Array-Destructuring:  
const koordinaten = [49.79, 9.93];  
const [lat, lng] = koordinaten; // lat = 49.79, lng = 9.93
```

Vorteil: Erspart unübersichtliches

```
const name = user.name; const stadt =  
user.stadt; .
```

Der Spread-Operator (...)

Vervielfältigt oder kombiniert Arrays und Objekte in einer neuen Struktur:

```
// 1. Arrays zusammenfügen / klonen:  
const primaer = ["Rot", "Blau"];  
const alleFarben = [...primaer, "Grün", "Gelb"]; // ["Rot", "Blau", "Grün", "Gelb"]  
  
// 2. Objekte erweitern / unveränderlich anpassen (Immutable Update):  
const basisConfig = { theme: "dark", fontSize: 16 };  
const userConfig = { ...basisConfig, fontSize: 18, sidebar: true };
```

**Unterschied: Spread (...) entpackt
Elemente; Rest (...) packt Elemente
in ein Array zusammen!**

Praxis-Check: Arrays & Moderne Methoden im Einsatz

- **Ausführbare Demodatei:** `Samples/JS/arrays-demo.html`
- **Gezeigte Kernkonzepte im Code:**
 1. **Grundlagen (Array-Literale `[]`):** Erstellung, 0-basierter Indexzugriff (`staedte[0]`) & `.length` - Eigenschaft.
 2. **`map()` (Transformieren):** Erzeugt neues Array mit berechneten Bruttopreisen (`netto * 1.19`).
 3. **`filter()` (Filtern):** Selektiert eine Untermenge von Objekten auf Basis von Bedingungen (`nettoPre < 50`).
 4. **`reduce()` (Aggregieren):** Summiert den gesamten Datenbestand schrittweise auf 1 Gesamtwert herunter.

Ausprobieren:

Öffne `Samples/JS/arrays-demo.html` im Browser und teste die interaktiven Buttons!

